Tutorial_4_Data_Preprocessing

March 7, 2025

Module 4: Data Preprocessing

The following tutorial contains Python examples for data preprocessing. You should refer to the "Data" chapter of the "Introduction to Data Mining" book (slides are available at https://www-users.cs.umn.edu/~kumar001/dmbook/index.php) to understand some of the concepts introduced in this tutorial. Data preprocessing consists of a broad set of techniques for cleaning, selecting, and transforming data to improve data mining analysis. Read the step-by-step instructions below carefully. To execute the code, click on the corresponding cell and press the SHIFT-ENTER keys simultaneously.

#Data Quality Issues

Poor data quality can have an adverse effect on data mining. Among the common data quality issues include noise, outliers, missing values, and duplicate data. This section presents examples of Python code to alleviate some of these data quality problems. We begin with an example dataset from the UCI machine learning repository containing information about breast cancer patients. We will first download the dataset using Pandas read_csv() function and display its first 5 data points.

[4]:
```
import pandas as pd
data = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-databases/
 ↪breast-cancer-wisconsin/breast-cancer-wisconsin.data', header=None)
data.columns = ['Sample code', 'Clump Thickness', 'Uniformity of Cell Size',␣
 ↪'Uniformity of Cell Shape',
                'Marginal Adhesion', 'Single Epithelial Cell Size', 'Bare␣
 ↪Nuclei', 'Bland Chromatin',
                'Normal Nucleoli', 'Mitoses','Class']

data = data.drop(['Sample code'],axis=1)
print('Number of instances = %d' % (data.shape[0]))
print('Number of attributes = %d' % (data.shape[1]))
data.head()
```

```
Number of instances = 699
Number of attributes = 10
```

[4]:

	Clump Thickness	Uniformity of Cell Size	Uniformity of Cell Shape	\
0	5	1	1	
1	5	4	4	
2	3	1	1	
3	6	8	8	

```
4                4                          1                           1

   Marginal Adhesion  Single Epithelial Cell Size Bare Nuclei  \
0                  1                            2           1
1                  5                            7          10
2                  1                            2           2
3                  1                            3           4
4                  3                            2           1

   Bland Chromatin  Normal Nucleoli  Mitoses  Class
0                3                1        1      2
1                3                2        1      2
2                3                1        1      2
3                3                7        1      2
4                3                1        1      2
```

Missing Values

It is not unusual for an object to be missing one or more attribute values. In some cases, the information was not collected; while in other cases, some attributes are inapplicable to the data instances. This section presents examples on the different approaches for handling missing values.

According to the description of the data (https://archive.ics.uci.edu/ml/datasets/breast+cancer+wisconsin+(origi the missing values are encoded as '?' in the original data. Our first task is to convert the missing values to NaNs. We can then count the number of missing values in each column of the data.

[7]:
```
import numpy as np

data = data.replace('?',np.NaN)

print('Number of instances = %d' % (data.shape[0]))
print('Number of attributes = %d' % (data.shape[1]))

print('Number of missing values:')
for col in data.columns:
    print('\t%s: %d' % (col,data[col].isna().sum()))
```

```
Number of instances = 699
Number of attributes = 10
Number of missing values:
        Clump Thickness: 0
        Uniformity of Cell Size: 0
        Uniformity of Cell Shape: 0
        Marginal Adhesion: 0
        Single Epithelial Cell Size: 0
        Bare Nuclei: 16
        Bland Chromatin: 0
        Normal Nucleoli: 0
        Mitoses: 0
```

```
Class: 0
```

Observe that only the 'Bare Nuclei' column contains missing values. In the following example, the missing values in the 'Bare Nuclei' column are replaced by the median value of that column. The values before and after replacement are shown for a subset of the data points.

'Code:'

[10]:
```
data2 = data['Bare Nuclei']

print('Before replacing missing values:')
print(data2[20:25])

data2 = pd.to_numeric(data2, errors='coerce')

data2 = data2.fillna(data2.median())

print('\nAfter replacing missing values:')
print(data2[20:25])
```

```
Before replacing missing values:
20     10
21      7
22      1
23    NaN
24      1
Name: Bare Nuclei, dtype: object

After replacing missing values:
20    10.0
21     7.0
22     1.0
23     1.0
24     1.0
Name: Bare Nuclei, dtype: float64
```

Instead of replacing the missing values, another common approach is to discard the data points that contain missing values. This can be easily accomplished by applying the dropna() function to the data frame.

'Code:'

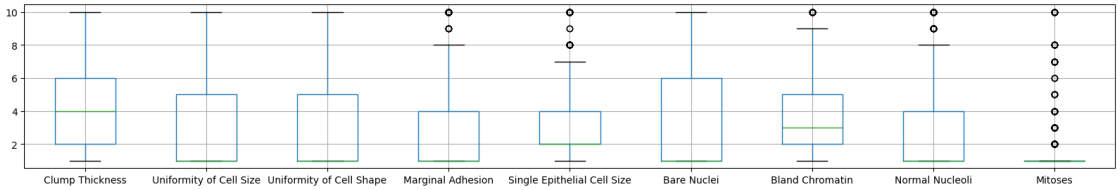
[13]:
```
print('Number of rows in original data = %d' % (data.shape[0]))

data2 = data.dropna()
print('Number of rows after discarding missing values = %d' % (data2.shape[0]))
```

```
Number of rows in original data = 699
Number of rows after discarding missing values = 683
```

Outliers

Outliers are data instances with characteristics that are considerably different from the rest of the dataset. In the example code below, we will draw a boxplot to identify the columns in the table that contain outliers. Note that the values in all columns (except for 'Bare Nuclei') are originally stored as 'int64' whereas the values in the 'Bare Nuclei' column are stored as string objects (since the column initially contains strings such as '?' for representing missing values). Thus, we must convert the column into numeric values first before creating the boxplot. Otherwise, the column will not be displayed when drawing the boxplot.

[16]:
```
%matplotlib inline
import matplotlib.pyplot as plt

data2 = data.drop(['Class'],axis=1)
data2['Bare Nuclei'] = pd.to_numeric(data2['Bare Nuclei'])
data2.boxplot(figsize=(20,3))
plt.show()
```



The boxplots suggest that only 5 of the columns (Marginal Adhesion, Single Epithetial Cell Size, Bland Cromatin, Normal Nucleoli, and Mitoses) contain abnormally high values. To discard the outliers, we can compute the Z-score for each attribute and remove those instances containing attributes with abnormally high or low Z-score (e.g., if Z > 3 or Z <= -3).

'Code:'

The following code shows the results of standardizing the columns of the data. Note that missing values (NaN) are not affected by the standardization process.

[19]:
```
Z = (data2-data2.mean())/data2.std()
Z[20:25]
```

[19]:

	Clump Thickness	Uniformity of Cell Size	Uniformity of Cell Shape	\
20	0.917080	-0.044070	-0.406284	
21	1.982519	0.611354	0.603167	
22	-0.503505	-0.699494	-0.742767	
23	1.272227	0.283642	0.603167	
24	-1.213798	-0.699494	-0.742767	

	Marginal Adhesion	Single Epithelial Cell Size	Bare Nuclei	\
20	2.519152	0.805662	1.771569	
21	0.067638	1.257272	0.948266	
22	-0.632794	-0.549168	-0.698341	

```
23          -0.632794                    -0.549168          NaN
24          -0.632794                    -0.549168    -0.698341

    Bland Chromatin  Normal Nucleoli   Mitoses
20         0.640688         0.371049  1.405526
21         1.460910         2.335921 -0.343666
22        -0.589645        -0.611387 -0.343666
23         1.460910         0.043570 -0.343666
24        -0.179534        -0.611387 -0.343666
```

'Code:'

The following code shows the results of discarding columns with Z > 3 or Z <= -3.

[22]:
```
print('Number of rows before discarding outliers = %d' % (Z.shape[0]))

Z2 = Z.loc[((Z > -3).sum(axis=1)==9) & ((Z <= 3).sum(axis=1)==9),:]
print('Number of rows after discarding missing values = %d' % (Z2.shape[0]))
```

```
Number of rows before discarding outliers = 699
Number of rows after discarding missing values = 632
```

Duplicate Data

Some datasets, especially those obtained by merging multiple data sources, may contain duplicates or near duplicate instances. The term deduplication is often used to refer to the process of dealing with duplicate data issues.

In the following example, we first check for duplicate instances in the breast cancer dataset.

[25]:
```
dups = data.duplicated()
print('Number of duplicate rows = %d' % (dups.sum()))
data.loc[[11,28]]
```

```
Number of duplicate rows = 236
```

[25]:
```
    Clump Thickness  Uniformity of Cell Size  Uniformity of Cell Shape  \
11                2                        1                         1
28                2                        1                         1

    Marginal Adhesion  Single Epithelial Cell Size Bare Nuclei  \
11                  1                            2           1
28                  1                            2           1

    Bland Chromatin  Normal Nucleoli  Mitoses  Class
11                2                1        1      2
28                2                1        1      2
```

The duplicated() function will return a Boolean array that indicates whether each row is a duplicate of a previous row in the table. The results suggest there are 236 duplicate rows in the breast cancer dataset. For example, the instance with row index 11 has identical attribute values as the

instance with row index 28. Although such duplicate rows may correspond to samples for different individuals, in this hypothetical example, we assume that the duplicates are samples taken from the same individual and illustrate below how to remove the duplicated rows.

'Code:'

[28]:
```
print('Number of rows before discarding duplicates = %d' % (data.shape[0]))
data2 = data.drop_duplicates()
print('Number of rows after discarding duplicates = %d' % (data2.shape[0]))
```

```
Number of rows before discarding duplicates = 699
Number of rows after discarding duplicates = 463
```

Shuffling Dataframes It is possible to shuffle.

[31]:
```
import os
import numpy as np
import pandas as pd

path = 'C:\\Users\\rampr\\OneDrive\\Documents\\' #data file can be dowloaded␣
 ↪from the link https://archive.ics.uci.edu/ml/datasets/
 ↪breast+cancer+wisconsin+(original)

filename_read = os.path.join(path,"auto-mpg.csv") #The file is available to␣
 ↪download from canvas in path Files --> Lab Help --> Labs --> data
df = pd.read_csv(filename_read, na_values=['NA','?'])

#np.random.seed(30) # Uncomment this line to get the same shuffle each time

df = df.reindex(np.random.permutation(df.index))
df.reset_index(inplace=True, drop=True)
# use inplace=False
df
```

[31]:

	mpg	cylinders	displacement	horsepower	weight	acceleration	year	\
0	15.0	8	429.0	198.0	4341	10.0	70	
1	31.0	4	76.0	52.0	1649	16.5	74	
2	30.5	4	98.0	63.0	2051	17.0	77	
3	16.0	8	304.0	150.0	3433	12.0	70	
4	15.0	8	383.0	170.0	3563	10.0	70	
..	…	…	…	…	…	…	…	…
393	24.0	6	200.0	81.0	3012	17.6	76	
394	24.0	4	116.0	75.0	2158	15.5	73	
395	11.0	8	429.0	208.0	4633	11.0	72	
396	26.8	6	173.0	115.0	2700	12.9	79	
397	23.0	4	115.0	95.0	2694	15.0	75	

	origin	name
0	1	ford galaxie 500

```
1         3              toyota corona
2         1         chevrolet chevette
3         1              amc rebel sst
4         1        dodge challenger se
..      …                         …
393       1              ford maverick
394       2                 opel manta
395       1            mercury marquis
396       1  oldsmobile omega brougham
397       2                 audi 100ls

[398 rows x 9 columns]
```

Sorting Dataframes It is possible to sort.

[34]:
```
df = df.sort_values(by='name',ascending=True)
df
```

[34]:
```
      mpg  cylinders  displacement  horsepower  weight  acceleration  year  \
258  13.0          8         360.0       175.0    3821          11.0    73
128  15.0          8         390.0       190.0    3850           8.5    70
145  17.0          8         304.0       150.0    3672          11.5    72
226  24.3          4         151.0        90.0    3003          20.1    80
315  19.4          6         232.0        90.0    3210          17.2    78
..    …         …           …          …      …           …    …
360  44.0          4          97.0        52.0    2130          24.6    82
325  41.5          4          98.0        76.0    2144          14.7    80
189  29.0          4          90.0        70.0    1937          14.2    76
342  44.3          4          90.0        48.0    2085          21.7    80
140  31.9          4          89.0        71.0    1925          14.0    79

     origin                     name
258       1  amc ambassador brougham
128       1       amc ambassador dpl
145       1       amc ambassador sst
226       1              amc concord
315       1              amc concord
..      …                       …
360       2               vw pickup
325       2               vw rabbit
189       2               vw rabbit
342       2     vw rabbit c (diesel)
140       2         vw rabbit custom

[398 rows x 9 columns]
```

[36]:
```
print("The first car is: {}".format(df['name'].iloc[0]))
```

```
The first car is: amc ambassador brougham
```

[38]:
```
print("The first car is: {}".format(df['name'].loc[0]))


#loc gets rows (or columns) with particular labels from the index.
#iloc gets rows (or columns) at particular positions in the index (so it only␣
 ↪takes integers).
```

```
The first car is: ford galaxie 500
```

Saving a Dataframe

The following code performs a shuffle and then saves a new copy.

[41]:
```
import os
import pandas as pd
import numpy as np

path = "C:\\Users\\rampr\\OneDrive\\Documents\\" #data file can be dowloaded␣
 ↪from the link https://archive.ics.uci.edu/ml/datasets/
 ↪breast+cancer+wisconsin+(original)

filename_read = os.path.join(path,"auto-mpg.csv") #The file is available to␣
 ↪download from canvas in path Files --> Lab Help --> Labs --> data
filename_write = os.path.join(path,"auto-mpg-shuffle.csv") #The file is␣
 ↪available to download from canvas in path Files --> Lab Help --> Labs -->␣
 ↪data
df = pd.read_csv(filename_read,na_values=['NA','?'])
df = df.reindex(np.random.permutation(df.index))
df.to_csv(filename_write,index=False)    # Specify index = false to not write␣
 ↪row numbers
print("Done")
```

```
Done
```

Dropping Fields

Some fields are of no value to the neural network and can be dropped. The following code removes the name column from the MPG dataset.

[44]:
```
import os
import pandas as pd
import numpy as np

path = "C:\\Users\\rampr\\OneDrive\\Documents\\" #data file can be dowloaded␣
 ↪from the link https://archive.ics.uci.edu/ml/datasets/
 ↪breast+cancer+wisconsin+(original)
```

```
filename_read = os.path.join(path,"auto-mpg.csv") #The file is available to␣
 ↪download from canvas in path Files --> Lab Help --> Labs --> data
df = pd.read_csv(filename_read,na_values=['NA','?'])

print("Before drop: {}".format(df.columns))
df.drop('name', axis=1, inplace=True)
print("After drop: {}".format(df.columns))
df[0:5]
```

```
Before drop: Index(['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
       'acceleration', 'year', 'origin', 'name'],
      dtype='object')
After drop: Index(['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
       'acceleration', 'year', 'origin'],
      dtype='object')
```

[44]:

	mpg	cylinders	displacement	horsepower	weight	acceleration	year	origin
0	18.0	8	307.0	130.0	3504	12.0	70	1
1	15.0	8	350.0	165.0	3693	11.5	70	1
2	18.0	8	318.0	150.0	3436	11.0	70	1
3	16.0	8	304.0	150.0	3433	12.0	70	1
4	17.0	8	302.0	140.0	3449	10.5	70	1

Calculated Fields

It is possible to add new fields to the dataframe that are calculated from the other fields. We can create a new column that gives the weight in kilograms. The equation to calculate a metric weight, given a weight in pounds is:

This can be used with the following Python code:

[47]:
```
import os
import pandas as pd
import numpy as np

path = "C:\\Users\\rampr\\OneDrive\\Documents\\" #data file can be dowloaded␣
 ↪from the link https://archive.ics.uci.edu/ml/datasets/
 ↪breast+cancer+wisconsin+(original)

filename_read = os.path.join(path,"auto-mpg.csv") #The file is available to␣
 ↪download from canvas in path Files --> Lab Help --> Labs --> data
df = pd.read_csv(filename_read,na_values=['NA','?'])
```

```
df.insert(1,'weight_kg',(df['weight']*0.45359237).astype(int))
df
```

[47]:

	mpg	weight_kg	cylinders	displacement	horsepower	weight	\
0	18.0	1589	8	307.0	130.0	3504	
1	15.0	1675	8	350.0	165.0	3693	
2	18.0	1558	8	318.0	150.0	3436	
3	16.0	1557	8	304.0	150.0	3433	
4	17.0	1564	8	302.0	140.0	3449	
..	…	…	…	…	…	…	
393	27.0	1265	4	140.0	86.0	2790	
394	44.0	966	4	97.0	52.0	2130	
395	32.0	1040	4	135.0	84.0	2295	
396	28.0	1190	4	120.0	79.0	2625	
397	31.0	1233	4	119.0	82.0	2720	

	acceleration	year	origin	name
0	12.0	70	1	chevrolet chevelle malibu
1	11.5	70	1	buick skylark 320
2	11.0	70	1	plymouth satellite
3	12.0	70	1	amc rebel sst
4	10.5	70	1	ford torino
..	…	…	…	…
393	15.6	82	1	ford mustang gl
394	24.6	82	2	vw pickup
395	11.6	82	1	dodge rampage
396	18.6	82	1	ford ranger
397	19.4	82	1	chevy s-10

[398 rows x 10 columns]

[50]:
```
import os
import pandas as pd
import numpy as np
from scipy.stats import zscore

path = "C:\\Users\\rampr\\OneDrive\\Documents\\" #data file can be dowloaded␣
 ↪from the link https://archive.ics.uci.edu/ml/datasets/
 ↪breast+cancer+wisconsin+(original)

filename_read = os.path.join(path,"auto-mpg.csv") #The file is available to␣
 ↪download from canvas in path Files --> Lab Help --> Labs --> data
df = pd.read_csv(filename_read,na_values=['NA','?'])
df['mpg'] = zscore(df['mpg'])
df
```

[50]:
```
           mpg  cylinders  displacement  horsepower  weight  acceleration  \
0    -0.706439          8         307.0       130.0    3504          12.0
1    -1.090751          8         350.0       165.0    3693          11.5
2    -0.706439          8         318.0       150.0    3436          11.0
3    -0.962647          8         304.0       150.0    3433          12.0
4    -0.834543          8         302.0       140.0    3449          10.5
..         …        …           …         …      …           …
393   0.446497          4         140.0        86.0    2790          15.6
394   2.624265          4          97.0        52.0    2130          24.6
395   1.087017          4         135.0        84.0    2295          11.6
396   0.574601          4         120.0        79.0    2625          18.6
397   0.958913          4         119.0        82.0    2720          19.4

     year  origin                       name
0      70       1  chevrolet chevelle malibu
1      70       1          buick skylark 320
2      70       1         plymouth satellite
3      70       1              amc rebel sst
4      70       1                ford torino
..    …     …                          …
393    82       1            ford mustang gl
394    82       2                  vw pickup
395    82       1              dodge rampage
396    82       1                ford ranger
397    82       1                 chevy s-10

[398 rows x 9 columns]
```

Missing Values

You can also simply drop any rows with any NA values. Another common practice is to replace missing values with the median value for that column. The following code replaces any NA values in horsepower with the median:

[53]:
```
import os
import pandas as pd
import numpy as np
from scipy.stats import zscore

path = "C:\\Users\\rampr\\OneDrive\\Documents\\" #data file can be dowloaded␣
 ↪from the link https://archive.ics.uci.edu/ml/datasets/
 ↪breast+cancer+wisconsin+(original)

filename_read = os.path.join(path,"auto-mpg.csv") #The file is available to␣
 ↪download from canvas in path Files --> Lab Help --> Labs --> data
df = pd.read_csv(filename_read,na_values=['NA','?'])
med = df['horsepower'].median()
df['horsepower'] = df['horsepower'].fillna(med)
```

```
# df = df.dropna() # you can also simply drop NA values
```

Concatenating Rows and Columns Rows and columns can be concatenated together to form new data frames.

[56]:
```
# Create a new dataframe from name and horsepower

import os
import pandas as pd
import numpy as np
from scipy.stats import zscore

path = "C:\\Users\\rampr\\OneDrive\\Documents\\" #data file can be dowloaded␣
 ↪from the link https://archive.ics.uci.edu/ml/datasets/
 ↪breast+cancer+wisconsin+(original)

filename_read = os.path.join(path,"auto-mpg.csv") #The file is available to␣
 ↪download from canvas in path Files --> Lab Help --> Labs --> data
df = pd.read_csv(filename_read,na_values=['NA','?'])
col_horsepower = df['horsepower']
col_name = df['name']
result = pd.concat([col_name,col_horsepower],axis=1)
result
```

[56]:

	name	horsepower
0	chevrolet chevelle malibu	130.0
1	buick skylark 320	165.0
2	plymouth satellite	150.0
3	amc rebel sst	150.0
4	ford torino	140.0
..	…	…
393	ford mustang gl	86.0
394	vw pickup	52.0
395	dodge rampage	84.0
396	ford ranger	79.0
397	chevy s-10	82.0

[398 rows x 2 columns]

[58]:
```
# Create a new dataframe from name and horsepower, but this time by row

import os
import pandas as pd
import numpy as np
from scipy.stats import zscore
```

```
path = "C:\\Users\\rampr\\OneDrive\\Documents\\" #data file can be dowloaded␣
 ↪from the link https://archive.ics.uci.edu/ml/datasets/
 ↪breast+cancer+wisconsin+(original)

filename_read = os.path.join(path,"auto-mpg.csv") #The file is available to␣
 ↪download from canvas in path Files --> Lab Help --> Labs --> data
df = pd.read_csv(filename_read,na_values=['NA','?'])
col_horsepower = df['horsepower']
col_name = df['name']
result = pd.concat([col_name,col_horsepower])
result
```

[58]:
```
0      chevrolet chevelle malibu
1              buick skylark 320
2             plymouth satellite
3                  amc rebel sst
4                    ford torino
                  …
393                          86.0
394                          52.0
395                          84.0
396                          79.0
397                          82.0
Length: 796, dtype: object
```

[11]:
```
import collections.abc
from sklearn import preprocessing
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import shutil
import os


# Encode text values to dummy variables(i.e. [1,0,0],[0,1,0],[0,0,1] for␣
 ↪red,green,blue)
def encode_text_dummy(df, name):
    dummies = pd.get_dummies(df[name])
    for x in dummies.columns:
        dummy_name = "{}-{}".format(name, x)
        df[dummy_name] = dummies[x]
    df.drop(name, axis=1, inplace=True)


# Encode text values to indexes(i.e. [1],[2],[3] for red,green,blue).
def encode_text_index(df, name):
    le = preprocessing.LabelEncoder()
```

```
    df[name] = le.fit_transform(df[name])
    return le.classes_


# Encode a numeric column as zscores
def encode_numeric_zscore(df, name, mean=None, sd=None):
    if mean is None:
        mean = df[name].mean()

    if sd is None:
        sd = df[name].std()

    df[name] = (df[name] - mean) / sd


# Convert all missing values in the specified column to the median
def missing_median(df, name):
    med = df[name].median()
    df[name] = df[name].fillna(med)


# Convert all missing values in the specified column to the default
def missing_default(df, name, default_value):
    df[name] = df[name].fillna(default_value)


# Convert a Pandas dataframe to the x,y inputs that TensorFlow needs
def to_xy(df, target):
    result = []
    for x in df.columns:
        if x != target:
            result.append(x)
    # find out the type of the target column.
    target_type = df[target].dtypes
    target_type = target_type[0] if isinstance(target_type, collections.abc.
 ↪Sequence) else target_type
    # Encode to int for classification, float otherwise. TensorFlow likes 32␣
 ↪bits.
    if target_type in (np.int64, np.int32):
        # Classification
        dummies = pd.get_dummies(df[target])
        return df[result].values.astype(np.float32), dummies.values.astype(np.
 ↪float32)
    else:
        # Regression
        return df[result].values.astype(np.float32), df[target].values.
 ↪astype(np.float32)
```

```
# Nicely formatted time string
def hms_string(sec_elapsed):
    h = int(sec_elapsed / (60 * 60))
    m = int((sec_elapsed % (60 * 60)) / 60)
    s = sec_elapsed % 60
    return "{}:{:>02}:{:>05.2f}".format(h, m, s)


# Regression chart.
def chart_regression(pred,y,sort=True):
    t = pd.DataFrame({'pred' : pred, 'y' : y.flatten()})
    if sort:
        t.sort_values(by=['y'],inplace=True)
    a = plt.plot(t['y'].tolist(),label='expected')
    b = plt.plot(t['pred'].tolist(),label='prediction')
    plt.ylabel('output')
    plt.legend()
    plt.show()

# Remove all rows where the specified column is +/- sd standard deviations
def remove_outliers(df, name, sd):
    drop_rows = df.index[(np.abs(df[name] - df[name].mean()) >= (sd * df[name].
 ↪std()))]
    df.drop(drop_rows, axis=0, inplace=True)


# Encode a column to a range between normalized_low and normalized_high.
def encode_numeric_range(df, name, normalized_low=-1, normalized_high=1,
                         data_low=None, data_high=None):
    if data_low is None:
        data_low = min(df[name])
        data_high = max(df[name])

    df[name] = ((df[name] - data_low) / (data_high - data_low)) *␣
 ↪(normalized_high - normalized_low) + normalized_low
```

Examples of label encoding, one hot encoding, and creating X/Y for TensorFlow

[64]:
```
df=pd.read_csv("iris.csv",na_values=['NA','?']) #The file is available to␣
 ↪download from canvas in path Files --> Lab Help --> Labs --> data
df
```

[64]:

	sepal_l	sepal_w	petal_l	petal_w	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa

```
3         4.6      3.1      1.5      0.2     Iris-setosa
4         5.0      3.6      1.4      0.2     Iris-setosa
..        …        …        …        …              …
145       6.7      3.0      5.2      2.3  Iris-virginica
146       6.3      2.5      5.0      1.9  Iris-virginica
147       6.5      3.0      5.2      2.0  Iris-virginica
148       6.2      3.4      5.4      2.3  Iris-virginica
149       5.9      3.0      5.1      1.8  Iris-virginica

[150 rows x 5 columns]
```

[66]:
```
encode_text_index(df,"species")   # label encoding
df
```

[66]:
```
     sepal_l  sepal_w  petal_l  petal_w  species
0        5.1      3.5      1.4      0.2        0
1        4.9      3.0      1.4      0.2        0
2        4.7      3.2      1.3      0.2        0
3        4.6      3.1      1.5      0.2        0
4        5.0      3.6      1.4      0.2        0
..       …        …        …        …          …
145      6.7      3.0      5.2      2.3        2
146      6.3      2.5      5.0      1.9        2
147      6.5      3.0      5.2      2.0        2
148      6.2      3.4      5.4      2.3        2
149      5.9      3.0      5.1      1.8        2

[150 rows x 5 columns]
```

[68]:
```
df=pd.read_csv("iris.csv",na_values=['NA','?']) #The file is available to␣
 ↪download from canvas in path Files --> Lab Help --> Labs --> data

encode_text_dummy(df,"species")   # One hot encoding
df
```

[68]:
```
     sepal_l  sepal_w  petal_l  petal_w  species-Iris-setosa  \
0        5.1      3.5      1.4      0.2                 True
1        4.9      3.0      1.4      0.2                 True
2        4.7      3.2      1.3      0.2                 True
3        4.6      3.1      1.5      0.2                 True
4        5.0      3.6      1.4      0.2                 True
..       …        …        …        …                   …
145      6.7      3.0      5.2      2.3                False
146      6.3      2.5      5.0      1.9                False
147      6.5      3.0      5.2      2.0                False
148      6.2      3.4      5.4      2.3                False
149      5.9      3.0      5.1      1.8                False
```

```
     species-Iris-versicolor  species-Iris-virginica
0                      False                   False
1                      False                   False
2                      False                   False
3                      False                   False
4                      False                   False
..                       …                       …
145                    False                    True
146                    False                    True
147                    False                    True
148                    False                    True
149                    False                    True

[150 rows x 7 columns]
```

Make sure you encode the lables first before you call to_xy()

[71]:
```
df=pd.read_csv("iris.csv",na_values=['NA','?']) #The file is available to␣
↪download from canvas in path Files --> Lab Help --> Labs --> data

encode_text_index(df,"species")    # encoding first before you call to_xy()

df
```

[71]:
```
     sepal_l  sepal_w  petal_l  petal_w  species
0        5.1      3.5      1.4      0.2        0
1        4.9      3.0      1.4      0.2        0
2        4.7      3.2      1.3      0.2        0
3        4.6      3.1      1.5      0.2        0
4        5.0      3.6      1.4      0.2        0
..       …      …        …        …        …
145      6.7      3.0      5.2      2.3        2
146      6.3      2.5      5.0      1.9        2
147      6.5      3.0      5.2      2.0        2
148      6.2      3.4      5.4      2.3        2
149      5.9      3.0      5.1      1.8        2

[150 rows x 5 columns]
```

[73]:
```
x,y = to_xy(df,"species")
```

[75]:
```
x
```

[75]:
```
array([[5.1, 3.5, 1.4, 0.2],
       [4.9, 3. , 1.4, 0.2],
       [4.7, 3.2, 1.3, 0.2],
       [4.6, 3.1, 1.5, 0.2],
```

```
[5. , 3.6, 1.4, 0.2],
[5.4, 3.9, 1.7, 0.4],
[4.6, 3.4, 1.4, 0.3],
[5. , 3.4, 1.5, 0.2],
[4.4, 2.9, 1.4, 0.2],
[4.9, 3.1, 1.5, 0.1],
[5.4, 3.7, 1.5, 0.2],
[4.8, 3.4, 1.6, 0.2],
[4.8, 3. , 1.4, 0.1],
[4.3, 3. , 1.1, 0.1],
[5.8, 4. , 1.2, 0.2],
[5.7, 4.4, 1.5, 0.4],
[5.4, 3.9, 1.3, 0.4],
[5.1, 3.5, 1.4, 0.3],
[5.7, 3.8, 1.7, 0.3],
[5.1, 3.8, 1.5, 0.3],
[5.4, 3.4, 1.7, 0.2],
[5.1, 3.7, 1.5, 0.4],
[4.6, 3.6, 1. , 0.2],
[5.1, 3.3, 1.7, 0.5],
[4.8, 3.4, 1.9, 0.2],
[5. , 3. , 1.6, 0.2],
[5. , 3.4, 1.6, 0.4],
[5.2, 3.5, 1.5, 0.2],
[5.2, 3.4, 1.4, 0.2],
[4.7, 3.2, 1.6, 0.2],
[4.8, 3.1, 1.6, 0.2],
[5.4, 3.4, 1.5, 0.4],
[5.2, 4.1, 1.5, 0.1],
[5.5, 4.2, 1.4, 0.2],
[4.9, 3.1, 1.5, 0.2],
[5. , 3.2, 1.2, 0.2],
[5.5, 3.5, 1.3, 0.2],
[4.9, 3.6, 1.4, 0.1],
[4.4, 3. , 1.3, 0.2],
[5.1, 3.4, 1.5, 0.2],
[5. , 3.5, 1.3, 0.3],
[4.5, 2.3, 1.3, 0.3],
[4.4, 3.2, 1.3, 0.2],
[5. , 3.5, 1.6, 0.6],
[5.1, 3.8, 1.9, 0.4],
[4.8, 3. , 1.4, 0.3],
[5.1, 3.8, 1.6, 0.2],
[4.6, 3.2, 1.4, 0.2],
[5.3, 3.7, 1.5, 0.2],
[5. , 3.3, 1.4, 0.2],
[7. , 3.2, 4.7, 1.4],
```

```
[6.4, 3.2, 4.5, 1.5],
[6.9, 3.1, 4.9, 1.5],
[5.5, 2.3, 4. , 1.3],
[6.5, 2.8, 4.6, 1.5],
[5.7, 2.8, 4.5, 1.3],
[6.3, 3.3, 4.7, 1.6],
[4.9, 2.4, 3.3, 1. ],
[6.6, 2.9, 4.6, 1.3],
[5.2, 2.7, 3.9, 1.4],
[5. , 2. , 3.5, 1. ],
[5.9, 3. , 4.2, 1.5],
[6. , 2.2, 4. , 1. ],
[6.1, 2.9, 4.7, 1.4],
[5.6, 2.9, 3.6, 1.3],
[6.7, 3.1, 4.4, 1.4],
[5.6, 3. , 4.5, 1.5],
[5.8, 2.7, 4.1, 1. ],
[6.2, 2.2, 4.5, 1.5],
[5.6, 2.5, 3.9, 1.1],
[5.9, 3.2, 4.8, 1.8],
[6.1, 2.8, 4. , 1.3],
[6.3, 2.5, 4.9, 1.5],
[6.1, 2.8, 4.7, 1.2],
[6.4, 2.9, 4.3, 1.3],
[6.6, 3. , 4.4, 1.4],
[6.8, 2.8, 4.8, 1.4],
[6.7, 3. , 5. , 1.7],
[6. , 2.9, 4.5, 1.5],
[5.7, 2.6, 3.5, 1. ],
[5.5, 2.4, 3.8, 1.1],
[5.5, 2.4, 3.7, 1. ],
[5.8, 2.7, 3.9, 1.2],
[6. , 2.7, 5.1, 1.6],
[5.4, 3. , 4.5, 1.5],
[6. , 3.4, 4.5, 1.6],
[6.7, 3.1, 4.7, 1.5],
[6.3, 2.3, 4.4, 1.3],
[5.6, 3. , 4.1, 1.3],
[5.5, 2.5, 4. , 1.3],
[5.5, 2.6, 4.4, 1.2],
[6.1, 3. , 4.6, 1.4],
[5.8, 2.6, 4. , 1.2],
[5. , 2.3, 3.3, 1. ],
[5.6, 2.7, 4.2, 1.3],
[5.7, 3. , 4.2, 1.2],
[5.7, 2.9, 4.2, 1.3],
[6.2, 2.9, 4.3, 1.3],
```

```
[5.1, 2.5, 3. , 1.1],
[5.7, 2.8, 4.1, 1.3],
[6.3, 3.3, 6. , 2.5],
[5.8, 2.7, 5.1, 1.9],
[7.1, 3. , 5.9, 2.1],
[6.3, 2.9, 5.6, 1.8],
[6.5, 3. , 5.8, 2.2],
[7.6, 3. , 6.6, 2.1],
[4.9, 2.5, 4.5, 1.7],
[7.3, 2.9, 6.3, 1.8],
[6.7, 2.5, 5.8, 1.8],
[7.2, 3.6, 6.1, 2.5],
[6.5, 3.2, 5.1, 2. ],
[6.4, 2.7, 5.3, 1.9],
[6.8, 3. , 5.5, 2.1],
[5.7, 2.5, 5. , 2. ],
[5.8, 2.8, 5.1, 2.4],
[6.4, 3.2, 5.3, 2.3],
[6.5, 3. , 5.5, 1.8],
[7.7, 3.8, 6.7, 2.2],
[7.7, 2.6, 6.9, 2.3],
[6. , 2.2, 5. , 1.5],
[6.9, 3.2, 5.7, 2.3],
[5.6, 2.8, 4.9, 2. ],
[7.7, 2.8, 6.7, 2. ],
[6.3, 2.7, 4.9, 1.8],
[6.7, 3.3, 5.7, 2.1],
[7.2, 3.2, 6. , 1.8],
[6.2, 2.8, 4.8, 1.8],
[6.1, 3. , 4.9, 1.8],
[6.4, 2.8, 5.6, 2.1],
[7.2, 3. , 5.8, 1.6],
[7.4, 2.8, 6.1, 1.9],
[7.9, 3.8, 6.4, 2. ],
[6.4, 2.8, 5.6, 2.2],
[6.3, 2.8, 5.1, 1.5],
[6.1, 2.6, 5.6, 1.4],
[7.7, 3. , 6.1, 2.3],
[6.3, 3.4, 5.6, 2.4],
[6.4, 3.1, 5.5, 1.8],
[6. , 3. , 4.8, 1.8],
[6.9, 3.1, 5.4, 2.1],
[6.7, 3.1, 5.6, 2.4],
[6.9, 3.1, 5.1, 2.3],
[5.8, 2.7, 5.1, 1.9],
[6.8, 3.2, 5.9, 2.3],
[6.7, 3.3, 5.7, 2.5],
```

```
       [6.7, 3. , 5.2, 2.3],
       [6.3, 2.5, 5. , 1.9],
       [6.5, 3. , 5.2, 2. ],
       [6.2, 3.4, 5.4, 2.3],
       [5.9, 3. , 5.1, 1.8]], dtype=float32)
```

[77]:
```
y
```

[77]:
```
array([[1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
```

```
[1., 0., 0.],
[1., 0., 0.],
[1., 0., 0.],
[1., 0., 0.],
[1., 0., 0.],
[1., 0., 0.],
[1., 0., 0.],
[1., 0., 0.],
[1., 0., 0.],
[1., 0., 0.],
[1., 0., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
```

```
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 1., 0.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
[0., 0., 1.],
```

```
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 0., 1.]], dtype=float32)
```

Example of Deal with Missing Values and Outliers

[80]:
```
path = "C:\\Users\\rampr\\OneDrive\\Documents\\" #data file can be dowloaded␣
 ↪from the link https://archive.ics.uci.edu/ml/datasets/
 ↪breast+cancer+wisconsin+(original)

filename_read = os.path.join(path,"auto-mpg.csv") #The file is available to␣
 ↪download from canvas in path Files --> Lab Help --> Labs --> data
df = pd.read_csv(filename_read,na_values=['NA','?'])

# Handle mising values in horsepower
missing_median(df, 'horsepower')
#df.drop('name', 1,inplace=True)

# Drop outliers in horsepower
print("Length before MPG outliers dropped: {}".format(len(df)))
remove_outliers(df,'mpg',2)
print("Length after MPG outliers dropped: {}".format(len(df)))
```

```
Length before MPG outliers dropped: 398
Length after MPG outliers dropped: 388
```

Training/Test Split

The code below performs a split of the MPG data into a training and validation set. The training set uses 80% of the data and the test(validation) set uses 20%.

The following image shows how a model is trained on 80% of the data and then validated against the remaining 20%.

[84]:
```
import pandas as pd
import io
import numpy as np
import os
from sklearn.model_selection import train_test_split


path = "C:\\Users\\rampr\\OneDrive\\Documents\\" #data file can be dowloaded␣
 ↪from the link https://archive.ics.uci.edu/ml/datasets/
 ↪breast+cancer+wisconsin+(original)

filename = os.path.join(path,"iris.csv") #The file is available to download␣
 ↪from canvas in path Files --> Lab Help --> Labs --> data
df = pd.read_csv(filename,na_values=['NA','?'])

df[0:5]
```

[84]:

	sepal_l	sepal_w	petal_l	petal_w	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

[86]:
```
from sklearn import preprocessing

le = preprocessing.LabelEncoder()
df['encoded_species'] = le.fit_transform(df['species'])

df[0:5]
```

[86]:

	sepal_l	sepal_w	petal_l	petal_w	species	encoded_species
0	5.1	3.5	1.4	0.2	Iris-setosa	0
1	4.9	3.0	1.4	0.2	Iris-setosa	0
2	4.7	3.2	1.3	0.2	Iris-setosa	0
3	4.6	3.1	1.5	0.2	Iris-setosa	0
4	5.0	3.6	1.4	0.2	Iris-setosa	0

[88]:
```
# Split into train/test
x_train, x_test, y_train, y_test = train_test_split(df[['sepal_l', 'sepal_w',␣
 ↪'petal_l', 'petal_w']], df['encoded_species'], test_size=0.25,␣
 ↪random_state=42)
```

[90]:
```
x_train.shape
```

[90]:
```
(112, 4)
```

[92]:
```
y_train.shape
```

[92]:
```
(112,)
```

[94]:
```
x_test.shape
```

[94]:
```
(38, 4)
```

[96]:
```
y_test.shape
```

[96]:
```
(38,)
```

Aggregation

Data aggregation is a preprocessing task where the values of two or more objects are combined into a single object. The motivation for aggregation includes (1) reducing the size of data to be processed, (2) changing the granularity of analysis (from fine-scale to coarser-scale), and (3) improving the stability of the data.

In the example below, we will use the daily precipitation time series data for a weather station located at Detroit Metro Airport. The raw data was obtained from the Climate Data Online website (https://www.ncdc.noaa.gov/cdo-web/). The daily precipitation time series will be compared against its monthly values.

The code below will load the precipitation time series data and draw a line plot of its daily time series.

[99]:
```
daily = pd.read_csv('DTW_prec.csv', header='infer') #The file is available to␣
 ↪download from canvas in path Files --> Lab Help --> Labs
daily.index = pd.to_datetime(daily['DATE'])
daily = daily['PRCP']
ax = daily.plot(kind='line',figsize=(15,3))
ax.set_title('Daily Precipitation (variance = %.4f)' % (daily.var()))
```

[99]:
```
Text(0.5, 1.0, 'Daily Precipitation (variance = 0.0530)')
```

[101]:
```
daily
```

[101]:
```
DATE
2001-01-01    0.00
2001-01-02    0.00
2001-01-03    0.00
2001-01-04    0.04
2001-01-05    0.14
              …
2017-12-27    0.00
2017-12-28    0.00
2017-12-29    0.00
2017-12-30    0.00
2017-12-31    0.00
```

```
Name: PRCP, Length: 6191, dtype: float64
```

Observe that the daily time series appear to be quite chaotic and varies significantly from one time step to another. The time series can be grouped and aggregated by month to obtain the total monthly precipitation values. The resulting time series appears to vary more smoothly compared to the daily time series.

'Code:'

[104]:
```
monthly = daily.groupby(pd.Grouper(freq='ME')).sum()
ax = monthly.plot(kind='line',figsize=(15,3))
ax.set_title('Monthly Precipitation (variance = %.4f)' % (monthly.var()))
```

[104]:
```
Text(0.5, 1.0, 'Monthly Precipitation (variance = 2.4241)')
```

In the example below, the daily precipitation time series are grouped and aggregated by year to obtain the annual precipitation values.

'Code:'

[107]:
```
annual = daily.groupby(pd.Grouper(freq='YE')).sum()
ax = annual.plot(kind='line',figsize=(15,3))
ax.set_title('Annual Precipitation (variance = %.4f)' % (annual.var()))
```

[107]:
```
Text(0.5, 1.0, 'Annual Precipitation (variance = 23.6997)')
```

Sampling

Sampling is an approach commonly used to facilitate (1) data reduction for exploratory data analysis and scaling up algorithms to big data applications and (2) quantifying uncertainties due to varying data distributions. There are various methods available for data sampling, such as sampling without replacement, where each selected instance is removed from the dataset, and sampling with replacement, where each selected instance is not removed, thus allowing it to be selected more than once in the sample.

In the example below, we will apply sampling with replacement and without replacement to the breast cancer dataset obtained from the UCI machine learning repository.

We initially display the first five records of the table.

[110]:
```
data.head()
```

[110]:

	Clump Thickness	Uniformity of Cell Size	Uniformity of Cell Shape	\
0	5	1	1	
1	5	4	4	
2	3	1	1	
3	6	8	8	
4	4	1	1	

	Marginal Adhesion	Single Epithelial Cell Size	Bare Nuclei	\
0	1	2	1	

```
1                   5                            7           10
2                   1                            2            2
3                   1                            3            4
4                   3                            2            1

   Bland Chromatin  Normal Nucleoli  Mitoses  Class
0                3                1        1      2
1                3                2        1      2
2                3                1        1      2
3                3                7        1      2
4                3                1        1      2
```

In the following code, a sample of size 3 is randomly selected (without replacement) from the original data.

'Code:'

[113]:
```
sample = data.sample(n=3)
sample
```

[113]:
```
     Clump Thickness  Uniformity of Cell Size  Uniformity of Cell Shape  \
663                1                        1                         3
226               10                        5                         7
105                7                        3                         4

     Marginal Adhesion  Single Epithelial Cell Size Bare Nuclei  \
663                  1                            2           1
226                  4                            4          10
105                  4                            3           3

     Bland Chromatin  Normal Nucleoli  Mitoses  Class
663                2                1        1      2
226                8                9        1      4
105                3                2        7      4
```

In the next example, we randomly select 1% of the data (without replacement) and display the selected samples. The random_state argument of the function specifies the seed value of the random number generator.

'Code:'

[116]:
```
sample = data.sample(frac=0.01, random_state=1)
sample
```

[116]:
```
     Clump Thickness  Uniformity of Cell Size  Uniformity of Cell Shape  \
584                5                        1                         1
417                1                        1                         1
606                4                        1                         1
349                4                        2                         3
```

```
134                3                          1                           1
502                4                          1                           1
117                4                          5                           5

     Marginal Adhesion  Single Epithelial Cell Size Bare Nuclei  \
584                  6                            3           1
417                  1                            2           1
606                  2                            2           1
349                  5                            3           8
134                  1                            3           1
502                  2                            2           1
117                 10                            4          10

     Bland Chromatin  Normal Nucleoli  Mitoses  Class
584                1                1        1      2
417                2                1        1      2
606                1                1        1      2
349                7                6        1      4
134                2                1        1      2
502                2                1        1      2
117                7                5        8      4
```

Finally, we perform a sampling with replacement to create a sample whose size is equal to 1% of the entire data. You should be able to observe duplicate instances in the sample by increasing the sample size.

'Code:'

[119]:
```
sample = data.sample(frac=0.01, replace=True, random_state=1)
sample
```

[119]:
```
     Clump Thickness  Uniformity of Cell Size  Uniformity of Cell Shape  \
37                 6                        2                         1
235                3                        1                         4
72                 1                        3                         3
645                3                        1                         1
144                2                        1                         1
129                1                        1                         1
583                3                        1                         1

     Marginal Adhesion  Single Epithelial Cell Size Bare Nuclei  \
37                   1                            1           1
235                  1                            2         NaN
72                   2                            2           1
645                  1                            2           1
144                  1                            2           1
129                  1                           10           1
583                  1                            2           1
```

```
     Bland Chromatin  Normal Nucleoli  Mitoses  Class
37                 7                1        1      2
235                3                1        1      2
72                 7                2        1      2
645                2                1        1      2
144                2                1        1      2
129                1                1        1      2
583                1                1        1      2
```

Discretization

Discretization is a data preprocessing step that is often used to transform a continuous-valued attribute to a categorical attribute. The example below illustrates two simple but widely-used unsupervised discretization methods (equal width and equal depth) applied to the 'Clump Thickness' attribute of the breast cancer dataset.

First, we plot a histogram that shows the distribution of the attribute values. The value_counts() function can also be applied to count the frequency of each attribute value.

'Code:'

[123]:
```
data['Clump Thickness'].hist(bins=10)
data['Clump Thickness'].value_counts(sort=False)
```

[123]:
```
Clump Thickness
5     130
3     108
6      34
4      80
8      46
1     145
2      50
7      23
10     69
9      14
Name: count, dtype: int64
```

For the equal width method, we can apply the cut() function to discretize the attribute into 4 bins of similar interval widths. The value_counts() function can be used to determine the number of instances in each bin.

'Code:'

[126]:
```
bins = pd.cut(data['Clump Thickness'],4)
bins.value_counts(sort=False)
```

[126]:
```
Clump Thickness
(0.991, 3.25]    303
(3.25, 5.5]      210
```

```
(5.5, 7.75]       57
(7.75, 10.0]     129
Name: count, dtype: int64
```

For the equal frequency method, the qcut() function can be used to partition the values into 4 bins such that each bin has nearly the same number of instances.

'Code:'

[129]:
```
bins = pd.qcut(data['Clump Thickness'],4)
bins.value_counts(sort=False)
```

[129]:
```
Clump Thickness
(0.999, 2.0]    195
(2.0, 4.0]      188
(4.0, 6.0]      164
(6.0, 10.0]     152
Name: count, dtype: int64
```

Principal Component Analysis

Principal component analysis (PCA) is a classical method for reducing the number of attributes in the data by projecting the data from its original high-dimensional space into a lower-dimensional space. The new attributes (also known as components) created by PCA have the following properties: (1) they are linear combinations of the original attributes, (2) they are orthogonal (perpendicular) to each other, and (3) they capture the maximum amount of variation in the data.

The example below illustrates the application of PCA to an image dataset. There are 16 RGB files, each of which has a size of 111 x 111 pixels. The example code below will read each image file and convert the RGB image into a 111 x 111 x 3 = 36963 feature values. This will create a data matrix of size 16 x 36963.

[132]:
```
%matplotlib inline
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import numpy as np

numImages = 16
fig = plt.figure(figsize=(7,7))
imgData = np.zeros(shape=(numImages,36963))

for i in range(1,numImages+1):
    filename = 'pics/Picture'+str(i)+'.jpg' #The file is available to download␣
 ↪from canvas in path Files --> Lab Help --> Labs
    img = mpimg.imread(filename)
    ax = fig.add_subplot(4,4,i)
    plt.imshow(img)
    plt.axis('off')
    ax.set_title(str(i))
```

```
    imgData[i-1] = np.array(img.flatten()).reshape(1,img.shape[0]*img.
↪shape[1]*img.shape[2])
```

Using PCA, the data matrix is projected to its first two principal components. The projected values of the original image data are stored in a pandas DataFrame object named projected.

'Code:'

[135]:
```
import pandas as pd
from sklearn.decomposition import PCA

numComponents = 2
pca = PCA(n_components=numComponents)
pca.fit(imgData)

projected = pca.transform(imgData)
projected = pd.
↪DataFrame(projected,columns=['pc1','pc2'],index=range(1,numImages+1))
projected['food'] = ['burger',␣
↪'burger','burger','burger','drink','drink','drink','drink',
                     'pasta', 'pasta', 'pasta', 'pasta', 'chicken', 'chicken',␣
↪'chicken', 'chicken']
projected
```

[135]:

	pc1	pc2	food
1	1576.633303	-6641.579951	burger
2	493.805733	-6396.931126	burger
3	-989.997999	-7235.877105	burger
4	-2189.908531	-9051.119853	burger
5	7843.026626	1061.056499	drink
6	8498.446537	5438.513971	drink
7	11181.714470	5319.181784	drink
8	6851.966063	-1124.359157	drink
9	-7635.165206	5043.477083	pasta
10	708.060147	528.715222	pasta
11	-7236.183483	5302.655454	pasta
12	-4417.291051	4660.244657	pasta
13	-11864.541342	-1472.672453	chicken
14	-76.475555	-1366.187141	chicken
15	7505.745161	1164.467461	chicken
16	-10249.834873	4770.414657	chicken

Finally, we draw a scatter plot to display the projected values. Observe that the images of burgers, drinks, and pastas are all projected to the same region. However, the images for fried chicken (shown as black squares in the diagram) are harder to discriminate.

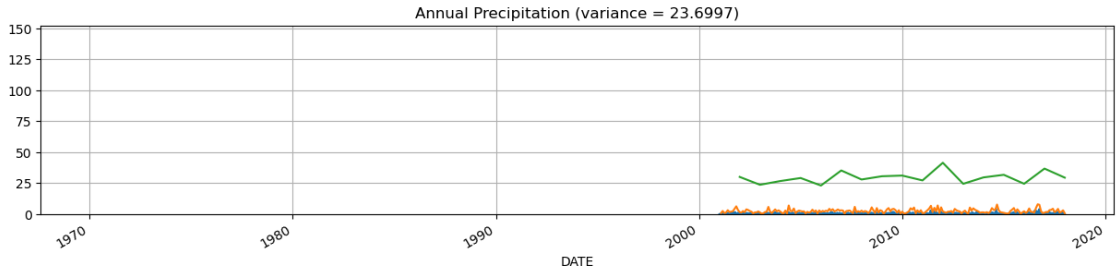
'Code:'

[138]:
```
import matplotlib.pyplot as plt

colors = {'burger':'b', 'drink':'r', 'pasta':'g', 'chicken':'k'}
markerTypes = {'burger':'+', 'drink':'x', 'pasta':'o', 'chicken':'s'}

for foodType in markerTypes:
    d = projected[projected['food']==foodType]
    plt.
 ↪scatter(d['pc1'],d['pc2'],c=colors[foodType],s=60,marker=markerTypes[foodType])
```

Feature Selection Techniques The goal of feature selection techniques in machine learning is to find the best set of features that allows one to build optimized models of studied phenomena.

Information Gain

[141]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
from sklearn.feature_selection import mutual_info_regression

df = pd.read_csv('Admission_Predict_Ver1.1_small_data_set_for_Linear_Regression.
 ↪csv') #The file is available to download from canvas in path Files --> Lab␣
 ↪Help --> Labs --> data --> Linear_Regression_data
X = df.iloc[:,1:-1]
Y = df.iloc[:,-1]
importances = mutual_info_regression(X, Y)
feat_importances = pd.Series(importances,X.columns)
feat_importances.plot(kind='barh',color = 'teal')
plt.show()
```



Chi-square Test The Chi-square test is used for categorical features in a dataset. We calculate Chi-square between each feature and the target and select the desired number of features with the best Chi-square scores. In order to correctly apply the chi-squared to test the relation between various features in the dataset and the target variable, the following conditions have to be met: the variables have to be categorical, sampled independently, and values should have an expected frequency greater than 5.

[144]:
```
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import chi2
from sklearn.preprocessing import LabelEncoder

X_Cat = X.astype(int)
le = LabelEncoder()
```

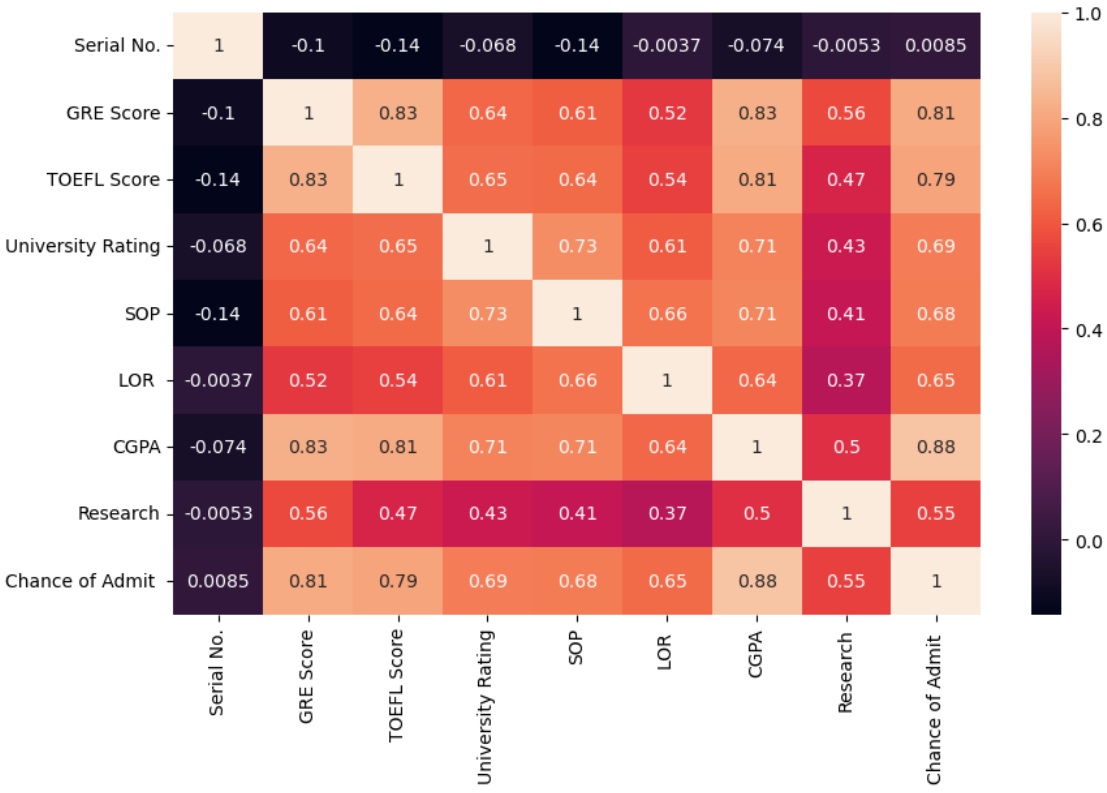
```
chi2_features = SelectKBest(chi2, k = 3)
Y = le.fit_transform(Y)
X_kbest_features = chi2_features.fit_transform(X_Cat,Y)

print('Original feature number:', X_Cat.shape[1])
print('Reduced feature number: ',X_kbest_features.shape[1])
```

```
Original feature number: 7
Reduced feature number:  3
```

Correlation Coefficient Correlation is a measure of the linear relationship between 2 or more variables. Through correlation, we can predict one variable from the other. The logic behind using correlation for feature selection is that good variables correlate highly with the target. Furthermore, variables should be correlated with the target but uncorrelated among themselves.

If two variables are correlated, we can predict one from the other. Therefore, if two features are correlated, the model only needs one, as the second does not add additional information. We will use the Pearson Correlation here.

[147]:
```
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline

cor = df.corr()

plt.figure(figsize=(10,6))
sns.heatmap(cor,annot= True)
plt.show()
```



Variance Threshold

The variance threshold is a simple baseline approach to feature selection. It removes all features whose variance doesn't meet some threshold. By default, it removes all zero-variance features, i.e., features with the same value in all samples. We assume that features with a higher variance may contain more useful information, but note that we are not taking the relationship between feature variables or feature and target variables into account, which is one of the drawbacks of filter methods.

The get_support returns a Boolean vector where True means the variable does not have zero variance.

[150]:
```
from sklearn.feature_selection import VarianceThreshold

X = globals()["X"]


v_threshold = VarianceThreshold(threshold = 0)
v_threshold.fit(X)
v_threshold.get_support()
```

[150]:
```
array([ True,  True,  True,  True,  True,  True,  True])
```

Mean Absolute Difference (MAD) 'The mean absolute difference (MAD) computes the absolute

difference from the mean value. The main difference between the variance and MAD measures is the absence of the square in the latter. The MAD, like the variance, is also a scaled variant.' This means that the higher the MAD, the higher the discriminatory power.

[153]:
```
mean_abs_diff = np.sum(np.abs(X -np.mean(X, axis=0)), axis=0)/X.shape[0]

plt.bar(np.arange(X.shape[1]),mean_abs_diff,color = 'teal')
```

[153]:
```
<BarContainer object of 7 artists>
```

Backward Elimination Techniques Importing required libraries

[156]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
from sklearn import preprocessing as prepr
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
import statsmodels.api as sm
from sklearn.metrics import mean_squared_error, r2_score
```

Import dataset and do necessary changes

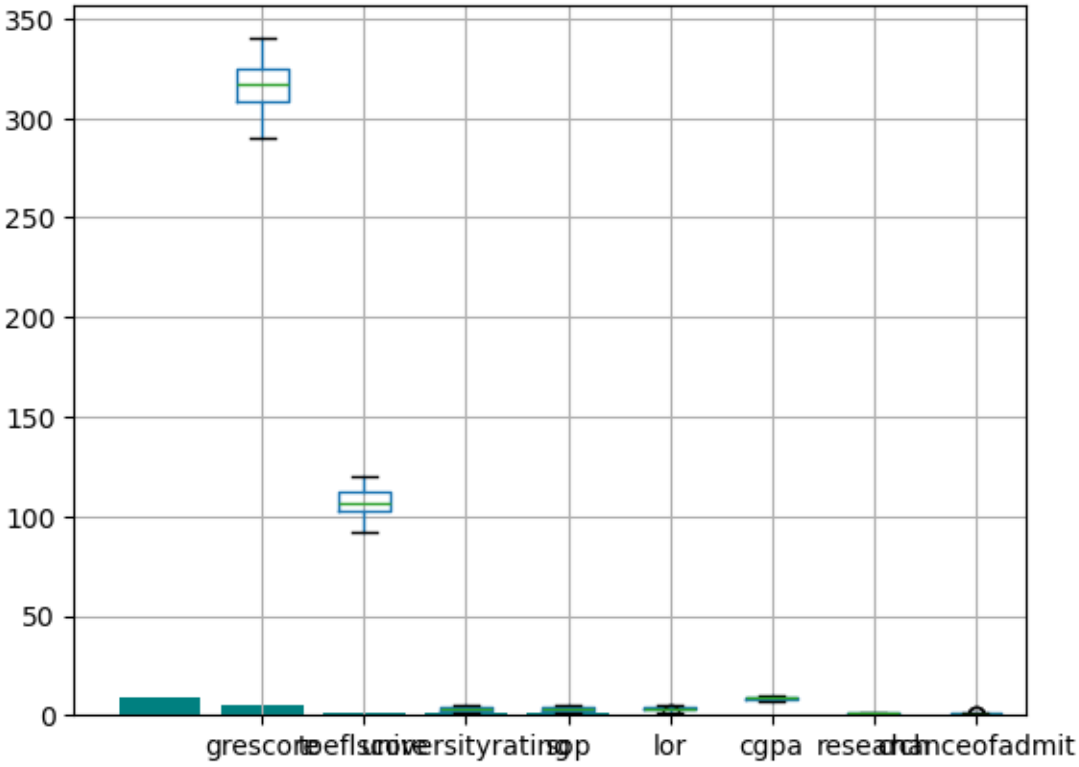
[159]:
```
df = pd.read_csv('Admission_Predict_Ver1.1_small_data_set_for_Linear_Regression.
 ↪csv') #The file is available to download from canvas in path Files --> Lab␣
 ↪Help --> Labs --> data --> Linear_Regression_data
print('Shape:{}'.format(df.shape))
print(df.head(10))
print(df.describe())
df.columns = [x.replace(' ', '').replace('.', '').lower() for x in list(df)]␣
 ↪#converts columnnames to lower single words
del df['serialno']
print('After deleting:\n', df.head(10))
```

```
Shape:(500, 9)
   Serial No.  GRE Score  TOEFL Score  University Rating  SOP  LOR   CGPA  \
0           1        337          118                  4  4.5   4.5  9.65
1           2        324          107                  4  4.0   4.5  8.87
2           3        316          104                  3  3.0   3.5  8.00
3           4        322          110                  3  3.5   2.5  8.67
4           5        314          103                  2  2.0   3.0  8.21
5           6        330          115                  5  4.5   3.0  9.34
6           7        321          109                  3  3.0   4.0  8.20
7           8        308          101                  2  3.0   4.0  7.90
8           9        302          102                  1  2.0   1.5  8.00
9          10        323          108                  3  3.5   3.0  8.60
```

```
   Research  Chance of Admit
0         1              0.92
1         1              0.76
2         1              0.72
3         1              0.80
4         0              0.65
5         1              0.90
6         1              0.75
7         0              0.68
8         0              0.50
9         0              0.45
       Serial No.   GRE Score  TOEFL Score  University Rating         SOP  \
count  500.000000  500.000000   500.000000         500.000000  500.000000
mean   250.500000  316.472000   107.192000           3.114000    3.374000
std    144.481833   11.295148     6.081868           1.143512    0.991004
min      1.000000  290.000000    92.000000           1.000000    1.000000
25%    125.750000  308.000000   103.000000           2.000000    2.500000
50%    250.500000  317.000000   107.000000           3.000000    3.500000
75%    375.250000  325.000000   112.000000           4.000000    4.000000
max    500.000000  340.000000   120.000000           5.000000    5.000000

            LOR         CGPA    Research  Chance of Admit
count  500.00000  500.000000  500.000000         500.00000
mean     3.48400    8.576440    0.560000           0.72174
std      0.92545    0.604813    0.496884           0.14114
min      1.00000    6.800000    0.000000           0.34000
25%      3.00000    8.127500    0.000000           0.63000
50%      3.50000    8.560000    1.000000           0.72000
75%      4.00000    9.040000    1.000000           0.82000
max      5.00000    9.920000    1.000000           0.97000
After deleting:
    grescore  toeflscore  universityrating  sop  lor  cgpa  research  \
0        337         118                 4  4.5  4.5  9.65         1
1        324         107                 4  4.0  4.5  8.87         1
2        316         104                 3  3.0  3.5  8.00         1
3        322         110                 3  3.5  2.5  8.67         1
4        314         103                 2  2.0  3.0  8.21         0
5        330         115                 5  4.5  3.0  9.34         1
6        321         109                 3  3.0  4.0  8.20         1
7        308         101                 2  3.0  4.0  7.90         0
8        302         102                 1  2.0  1.5  8.00         0
9        323         108                 3  3.5  3.0  8.60         0

   chanceofadmit
0           0.92
1           0.76
2           0.72
3           0.80
```

```
4           0.65
5           0.90
6           0.75
7           0.68
8           0.50
9           0.45
```

[161]:
```
df.boxplot(figsize=(10,8))
plt.show()
```



Standardizing the entire dataset using Min-Max scaling

[164]:
```
cols = list(df)
scaler = prepr.MinMaxScaler()
scaled_df = scaler.fit_transform(df)
scaled_df = pd.DataFrame(scaled_df, columns=cols)
print(scaled_df.head(10))
print(scaled_df.describe())
```

```
   grescore  toeflscore  universityrating     sop     lor      cgpa  research  \
0      0.94    0.928571              0.75  0.875  0.875  0.913462       1.0
1      0.68    0.535714              0.75  0.750  0.875  0.663462       1.0
2      0.52    0.428571              0.50  0.500  0.625  0.384615       1.0
```

```
3      0.64    0.642857             0.50  0.625  0.375  0.599359       1.0
4      0.48    0.392857             0.25  0.250  0.500  0.451923       0.0
5      0.80    0.821429             1.00  0.875  0.500  0.814103       1.0
6      0.62    0.607143             0.50  0.500  0.750  0.448718       1.0
7      0.36    0.321429             0.25  0.500  0.750  0.352564       0.0
8      0.24    0.357143             0.00  0.250  0.125  0.384615       0.0
9      0.66    0.571429             0.50  0.625  0.500  0.576923       0.0

   chanceofadmit
0       0.920635
1       0.666667
2       0.603175
3       0.730159
4       0.492063
5       0.888889
6       0.650794
7       0.539683
8       0.253968
9       0.174603
          grescore  toeflscore  universityrating         sop         lor  \
count  500.000000  500.000000        500.000000  500.000000  500.000000
mean     0.529440    0.542571          0.528500    0.593500    0.621000
std      0.225903    0.217210          0.285878    0.247751    0.231362
min      0.000000    0.000000          0.000000    0.000000    0.000000
25%      0.360000    0.392857          0.250000    0.375000    0.500000
50%      0.540000    0.535714          0.500000    0.625000    0.625000
75%      0.700000    0.714286          0.750000    0.750000    0.750000
max      1.000000    1.000000          1.000000    1.000000    1.000000

              cgpa    research  chanceofadmit
count  500.000000  500.000000     500.000000
mean     0.569372    0.560000       0.605937
std      0.193850    0.496884       0.224032
min      0.000000    0.000000       0.000000
25%      0.425481    0.000000       0.460317
50%      0.564103    1.000000       0.603175
75%      0.717949    1.000000       0.761905
max      1.000000    1.000000       1.000000
```

Pairplot

This is a scatter plot of all the attributes on both x and y axes. 'chanceofadmit' is the dependent variable and the plot clearly shows that there are few attributes that have a linear relation with the dependent attribute. So we can move ahead with linear regression.

[168]:
```
sns.pairplot(scaled_df)
```

[168]:
```
<seaborn.axisgrid.PairGrid at 0x1a8eb2a6540>
```

Creating X and Y for the linear regression equation ($Y = a + \beta X_1 + \beta X_2 + \ldots + \beta X_n$) where Y is the dependent variable and Xs are the independent variables

[170]:
```
cols = list(scaled_df)
X = scaled_df.iloc[:, :-1]
y = scaled_df[cols[-1]]
```

Using statsmodels provided Linear Regression Model

OLS is the function used to create a linear model. This model defines the linear regression formula as $Y = aX_0 + \beta X_1 + \beta X_2 + \ldots + \beta X_n$.

So create a new attribute with all ones and append it to the start of the dataframe. This will serve as X_0

[173]:
```
X = np.append(arr=np.ones([len(scaled_df.index), 1]).astype(int), values=X,␣
↪axis=1)
options = X[:, [0, 1, 2, 3, 4, 5, 6, 7]]
lm_be = sm.OLS(endog=y, exog=options).fit()
print(lm_be.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:          chanceofadmit   R-squared:                       0.822
Model:                            OLS   Adj. R-squared:                  0.819
Method:                 Least Squares   F-statistic:                     324.4
Date:                Fri, 07 Mar 2025   Prob (F-statistic):          8.21e-180
Time:                        21:29:32   Log-Likelihood:                 470.37
No. Observations:                 500   AIC:                            -924.7
Df Residuals:                     492   BIC:                            -891.0
Df Model:                           7
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.0130      0.014      0.902      0.367      -0.015       0.041
x1             0.1475      0.040      3.700      0.000       0.069       0.226
x2             0.1235      0.039      3.184      0.002       0.047       0.200
x3             0.0377      0.024      1.563      0.119      -0.010       0.085
x4             0.0101      0.029      0.348      0.728      -0.047       0.067
x5             0.1070      0.026      4.074      0.000       0.055       0.159
x6             0.5863      0.048     12.198      0.000       0.492       0.681
x7             0.0386      0.010      3.680      0.000       0.018       0.059
==============================================================================
Omnibus:                      112.770   Durbin-Watson:                   0.796
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              262.104
Skew:                          -1.160   Prob(JB):                     1.22e-57
Kurtosis:                       5.684   Cond. No.                         23.4
==============================================================================
```

```
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```

[174]:
```
#Remove attribute 4 (sop) as it has the highest p-value and rewrite the formula
options= X[:, [0, 1, 2, 3, 5, 6, 7]]
lm_be = sm.OLS(endog=y, exog=options).fit()
lm_be.summary()
```

[174]:

Dep. Variable:	chanceofadmit	**R-squared:**	0.822
Model:	OLS	**Adj. R-squared:**	0.820
Method:	Least Squares	**F-statistic:**	379.1
Date:	Fri, 07 Mar 2025	**Prob (F-statistic):**	4.29e-181
Time:	21:29:32	**Log-Likelihood:**	470.31
No. Observations:	500	**AIC:**	-926.6
Df Residuals:	493	**BIC:**	-897.1
Df Model:	6		
Covariance Type:	nonrobust		

	coef	**std err**	**t**	**P> |t|**	**[0.025**	**0.975]**
const	0.0135	0.014	0.935	0.350	-0.015	0.042
x1	0.1471	0.040	3.694	0.000	0.069	0.225
x2	0.1248	0.039	3.236	0.001	0.049	0.201
x3	0.0408	0.022	1.820	0.069	-0.003	0.085
x4	0.1098	0.025	4.380	0.000	0.061	0.159
x5	0.5893	0.047	12.481	0.000	0.497	0.682
x6	0.0387	0.010	3.691	0.000	0.018	0.059

Omnibus:	111.782	**Durbin-Watson:**	0.800
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	258.656
Skew:	-1.152	**Prob(JB):**	6.82e-57
Kurtosis:	5.667	**Cond. No.**	21.8

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[175]:
```
#Remove attribute 3 (universityrating) as it has the highest p-value and␣
  ↪rewrite the formula
options = X[:, [0, 1, 2, 5, 6, 7]]
lm_be = sm.OLS(endog=y, exog=options).fit()
lm_be.summary()
```

[175]:

Dep. Variable:	chanceofadmit	**R-squared:**	0.821
Model:	OLS	**Adj. R-squared:**	0.819
Method:	Least Squares	**F-statistic:**	452.1
Date:	Fri, 07 Mar 2025	**Prob (F-statistic):**	9.97e-182
Time:	21:29:32	**Log-Likelihood:**	468.63
No. Observations:	500	**AIC:**	-925.3
Df Residuals:	494	**BIC:**	-900.0
Df Model:	5		
Covariance Type:	nonrobust		

	coef	**std err**	**t**	**P> |t|**	**[0.025**	**0.975]**
const	0.0085	0.014	0.598	0.550	-0.019	0.036
x1	0.1499	0.040	3.760	0.000	0.072	0.228
x2	0.1341	0.038	3.501	0.001	0.059	0.209
x3	0.1227	0.024	5.092	0.000	0.075	0.170
x4	0.6090	0.046	13.221	0.000	0.519	0.700
x5	0.0399	0.010	3.814	0.000	0.019	0.061

Omnibus:	109.027	**Durbin-Watson:**	0.800
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	248.874
Skew:	-1.130	**Prob(JB):**	9.07e-55
Kurtosis:	5.615	**Cond. No.**	20.4

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Create train and test set from the attributes that are not eliminated by backward elimination

[177]:
```
X_train, X_test, y_train, y_test = train_test_split(options, y, test_size=0.2,␣
↪random_state=123)
```

Create a regular Linear Regression model

[179]:
```
lm = LinearRegression().fit(X_train, y_train)
```

Predicting

[181]:
```
#Predict
y_pred = lm.predict(X_test)
```

[ ]:
```
Display Result
```

[191]:
```
final_df = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})
print(final_df.head(10))
```

```
       Actual  Predicted
229  0.761905   0.740063
337  0.952381   0.941972
327  0.555556   0.301407
416  0.492063   0.446706
306  0.714286   0.761012
```

```
131  0.682540   0.563316
5    0.888889   0.844722
431  0.619048   0.699565
434  0.476190   0.442705
134  0.873016   0.858961
```

Calculate accuracy

[194]:
```
print("Mean squared error: %.2f" % mean_squared_error(y_test, y_pred))
print('Variance score: %.2f' % r2_score(y_test, y_pred))
```

```
Mean squared error: 0.01
Variance score: 0.78
```

[ ]:

[ ]:

[ ]: